

HeteroMRTA-Priority: Teaching Multi-Robot Systems to Eat The Frog

Alina Wang MRSD alinaw@andrew.cmu.edu		Esme Rubinstein RI PhD erubinst@andrew.cmu.edu
Minsik Jeon MSR minsikj@andrew.cmu.edu	Shunxuan Wang MRSD shunxuaw@andrew.cmu.edu	Itak Choi MSME itakc@andrew.cmu.edu

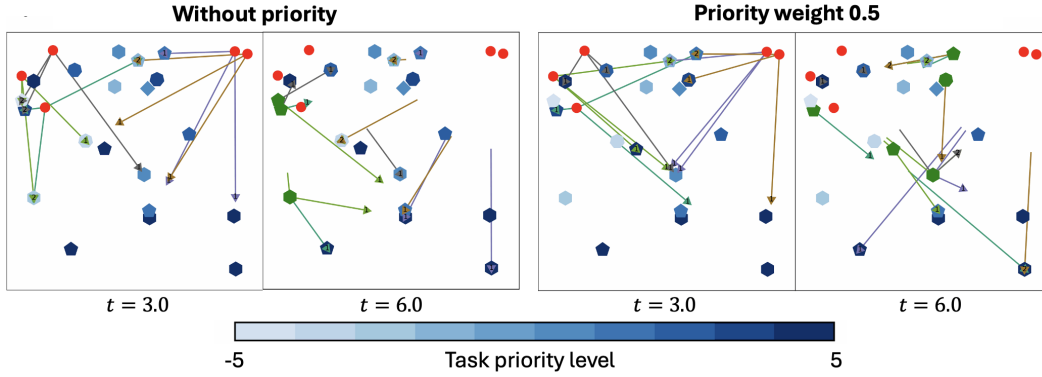


Figure 1: The left panels display the baseline agent, while the right panels depict our priority-based agent. Arrows indicate agent trajectories, red circles denote species depots, and tasks are represented as polygons (darker colors indicate higher priority). Notably, our agent demonstrates effective prioritization by addressing the highest-priority tasks immediately.

Abstract: With rapid robotics development, efficient multi-robot task allocation (MRTA) and scheduling are becoming increasingly critical to enable coordination between heterogeneous robotics agents operating in dynamic environments. Traditional MRTA solutions, such as mixed integer programming, heuristics scheduling, game theory approaches, are adept at complex scenarios, like task priorities and precedence relationships, but often struggle to maintain scalable performance. In this work, we want to build upon reinforcement learning method to solve ST-MR-TA while adding priority queue and precedence relationships. Our method modifies the reward structure to penalize failures when handling high-priority tasks and encourages agents to re-order tasks based on precedence weighting. Through simulations, we demonstrate priority fulfillment and task compliance while maintaining reasonable makespan and scalability when compared to traditional solutions and the learning-based approach on which this work builds.

Keywords: Multi-Robot, Reinforcement Learning, Task Allocation, Priority

1 Introduction

The robotics era is changing from integrating sensors into products to composing entire systems out of modular robots. This rapid growth implies an increasing need for scalable and decentralized coordination methods to allocate work across ground, aerial, marine, and all types of robotics platforms in the near future. Previous work on multi-robot task allocation with a focus on single-task robots,

multi-robot tasks, time-extended assignment (ST-MR-TA) utilizes methods like heuristics and game theory ideas Bischoff et al. [1], Li et al. [2, 3]. Although these are very insightful, their scalability falls short compared to learning-based solutions, like [4, 5, 6]. Therefore, we deepen our research in learning-based MRTA to exploit its advantages in scalability, handling of complex and dynamic task allocation, and reduction in makespan in large-scale scenarios.

Aside from reproducing the learning-based MRTA results from Dai et al [7], we extend their work by experimenting with ways to represent task priority and the scalability of the system afterwards. In realistic scenarios, tasks often have an underlying priority assigned to them. For example, ground robots carrying patients should have the right of the road, compared to a ground robot that clears debris. Traditional scheduling approaches like MIP and heuristic methods are good at capturing complex precedence constraints but often fail to scale, suffering from high computation time or only finding sub-optimal solutions in large-scale problems Leahy et al. [8], Zhang et al. [9]. Therefore, establishing a balance between the scalability of learning-based approaches and the performance requirements of complex task constraints is an important area to address.

Therefore, we extend the learning-based MRTA research, specifically in the ST-MR-TA area, by integrating a priority-weighted term into the reward function. By modifying the objective function to prioritize urgent tasks, our research demonstrates improved priority fulfillment in large heterogeneous teams of up to 150 agents.

2 Related Works

2.1 Multi-Robot Task Allocation and Scheduling

Multi-robot task allocation (MRTA) problems are commonly classified along three axes: single- vs. multi-task robots, single- vs. multi-robot tasks, and instantaneous vs. time-extended assignment [10]. Classical routing problems such as the Traveling Salesman Problem and the Vehicle Routing Problem fall in the ST-SR-TA category and have been extensively studied by Bektas [11] and Braekers et al.[12] However, these formulations typically assume homogeneous agents and tasks that can be completed by a single robot, and thus do not capture tight multi-robot cooperation.

Motivated by heterogeneous robot teams deployed in complex missions such as construction, logistics, and agriculture, recent work increasingly focuses on the ST-MR-TA setting, where each task may require a coalition of robots with complementary skills and allocation and scheduling are tightly coupled [13, 14].

Centralized optimization-based approaches formulate MRTA with scheduling as large mixed-integer programs. Fu et al. present a robust task scheduling framework that jointly optimizes task decomposition, coalition formation, and timing under capability uncertainty [15]. Similarly, Korsah et al. use MIP to generate multi-robot plans with cross-schedule dependencies in their xBots framework [16]. These methods can provide high-quality or even optimal solutions but often incur prohibitive computation times as the number of robots, tasks, and dependencies grows.

To improve scalability, several works replace exact optimization with metaheuristics. For instance, Babincsak et al. design an ant-colony-optimization-based algorithm for heterogeneous coalition formation and scheduling with multi-skilled robots [17]. Such evolutionary approaches can generate good solutions quickly but still tend to be tuned for moderate problem sizes, and they usually support only limited forms of cooperation structure.

More recently, deep reinforcement learning has been explored as an alternative for MRTA with scheduling, shifting most of the computational burden to offline training while enabling fast decentralized decision-making at deployment. Dai et al. propose an attention-based RL framework that learns a cooperative policy for heterogeneous teams and implicitly handles coalition formation and reactive scheduling [7].

Our work builds on this line of research, but further incorporates task priorities into the allocation-scheduling process, as discussed in the following section.

2.2 Priority-Driven Task Scheduling

Priority-driven task scheduling has roots in real-time systems, where task priorities encode urgency or criticality and the scheduler always dispatches the highest-priority ready job. Classical work such as Goossens et al. [18] shows that carefully designed priority-based algorithms can guarantee schedulability for task sets that defeat more general policies like earliest-deadline-first, highlighting both the simplicity and effectiveness of priority as a scheduling primitive.

At the level of multi-robot missions, priority-driven scheduling is used to ensure urgent or safety-critical tasks are served ahead of routine operations. Kalempa et al. introduces MRPF [19], which combines priority policies with preemptive task switching and fault recovery, allowing higher-priority logistics processes to interrupt ongoing low-priority jobs. In industrial multi-robot teams with tightly coupled temporal and spatial constraints, Gombolay et al. propose the Tercio framework [20]. Inspired by real-time processor scheduling, Tercio uses dynamic priority heuristics to guide a fast multi-agent task sequencer toward low-makespan schedules under complex temporal constraints. These approaches demonstrate priority rules can greatly improve responsiveness and robustness in multi-robot task scheduling, but they typically rely on centralized solvers and hand-crafted priority heuristics. They are less suited to highly dynamic environments with large heterogeneous teams.

More recently, learning-based and predictive methods have started to treat priority as an explicit optimization objective. Zhang et al. presents a deep reinforcement learning framework [21] where their task priorities are integrated into the reward function and an attention-based actor-critic architecture generates online schedules, which accelerate completion of high-priority tasks while adapting to changing task sets. Gao et al. proposes Priorest [22], using a neural model to predict future events and slack times so that the scheduler can proactively favor tasks which are more urgent or at risk of missing deadlines. While these works highlight the benefits of coupling priority models with data-driven scheduling, they are often tailored to specific domains and still assume a largely centralized coordination structure. In contrast, our work builds on RL-based heterogeneous multi-robot task allocation and scheduling from Dai et al [7], where the learned policy can be generalized to arbitrary scenarios. We elevate task priority to a first-class element within the learned policy, preserving scalability and the ability to handle complex cooperative tasks.

3 Methodology

3.1 Problem Formulation & Prior Approach

We adopt the *species-traits* model [7, 15] which represents a multi robot system as a community of agents where each agent belongs to a species that has a unique set of skills. Considering a set of agents $N = a_1, \dots, a_{k_n}$, agents of the same species start at the same depot, complete all tasks $M = \{m_1, \dots, m_{k_m}\}$ distributed in a 2D domain, and then return to their depot.

Each task m_j requires a set of skills that are needed for completion and an execution time t_{m_j} . A task cannot begin until all agents for each of the skills arrives at the task location, and they must remain there for the entire execution duration for the task to be completed. All tasks and depots are represented on a graph where the edges are the Euclidean distance between the locations. The objective is to minimize the makespan, defined as the maximum total working time among all agents to finish entire tasks.

In [7], the above problem is addressed by casting the MRTA problem into a reinforcement learning problem with the following reward function:

$$R(\Phi) = -T - W \quad (1)$$

Here, T denotes the makespan and W refers to the average wasted availability ratio, which is the ratio of skills of the agents present on a task that are not required for that task. Maximizing Eq 1 effectively minimizes the makespan, ensuring that agents complete all tasks as quickly as possible.

Subtracting W discourages skill redundancy, while simultaneously reducing the likelihood of deadlocks (when all agents cannot complete their current tasks because they are waiting for each other). A more detailed formulation of the problem and objective can be found in [7].

3.2 Problem Statement & Our Approach

While the prior formulation enables efficient task allocation, it does not incorporate task priority, which may result in high-priority tasks being completed late. Our goal is to incorporate task priority within the allocation policy, encouraging the agents to execute tasks in descending priority order while still minimizing overall makespan.

To model priority, we randomly assign each task m_j an integer-valued task priority $p_j \in [-5, 5]$ sampled uniformly during training, where larger values correspond to higher priority. We extend the original MRTA RL formulation by introducing a priority reward term P into Eq 1:

$$R(\Phi) = -T - W + \omega_P P, \quad P = \sum_{j=1}^{k_n} \frac{k_n - o(j)}{k_n} p_j, \quad (2)$$

where ω_P is the priority weight assigned to the priority reward and $o(j)$ denotes the completion order of task m_j among the k_n tasks. Maximizing the priority reward encourages task with higher priority values to be completed earlier (i.e., with smaller $o(j)$). As a result, the above formulation of the reward function simultaneously minimizes makespan, while prioritizing more important tasks.

Following [7], we train our policy using the REINFORCE [23] algorithm with baseline inspired by POMO [24]. Instead of choosing different first action in the traveling salesperson problem during the task routing and calculating the average reward for all solutions like in POMO, we run the same episode multiple times and calculate the average reward.

4 Experiment

We conduct several experiments for small and large MA-AT problems. We compare our updated priority model to the Dai’s model without priority and report the effects on makespan, computation time, and waiting time. As in [7], we train our policy on randomly generated instances for each episode. We assume all agents move at 0.2 velocity. We train three policies to test varying degrees of priority. First, we train a model with no priority, matching the ideas solely in [7]. We also train a model with priority weight 0.1 and 0.5 within our reward.

Table 1: MA-AT Scenarios

MA-AT	Model	Priority Reward	Makespan	AWT
$k_m = 20,$ $k_n = 15$	RL(0P)	-2.972 (± 3.288)	23.466 (± 6.107)	3.305 (± 3.030)
	RL(0.1P)	12.705 (± 4.207)	23.067 (± 2.498)	3.461 (± 0.734)
	RL(0.5P)	15.353 (± 4.243)	23.483 (± 2.597)	3.374 (± 1.900)
$k_m = 500,$ $k_n = 50$	RL(0P)	-172.0175 (± 49.906)	106.665 (± 30.395)	26.466 (± 18.642)
	RL(0.1P)	328.784 (± 96.029)	104.180 (± 25.787)	24.768 (± 15.678)
	RL(0.5P)	415.147 (± 115.864)	106.0518 (± 22.533)	23.887 (± 14.162)
$k_m = 500,$ $k_n = 150$	RL(0P)	-160.094 (± 57.448)	41.967 (± 9.148)	7.053 (± 4.098)
	RL(0.1P)	333.970 (± 96.488)	39.939 (± 7.122)	6.572 (± 3.711)
	RL(0.5P)	414.947 (± 115.786)	40.556 (± 7.445)	6.273 (± 2.957)

For each model, we test on a smaller scale problem with 50 episodes with 15 agents (k_n) and 20 tasks (k_m). We also test on two larger scale problems: 50 agents and 500 tasks and 150 agents and 500 tasks. For each test set, we calculate the average makespan (overall task completion time), priority reward, and average wait time (the average duration each agent wastes on waiting for others at tasks). We average over the 50 episodes for the results in Table 1.

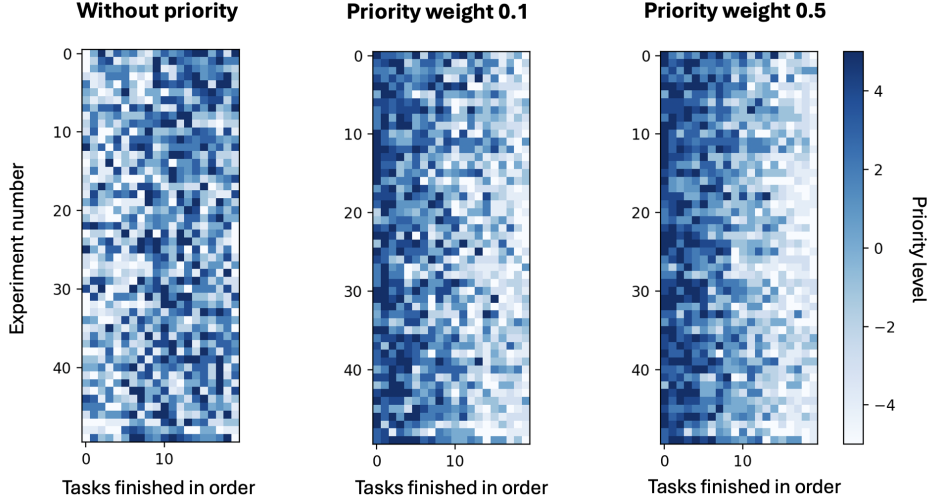


Figure 2: Priority values of tasks in order they are finished. The vertical axis represents the number of experiments with different environment. With higher weight of the priority reward, the agent shows clearer trend in attempting high-priority tasks first.

Table 2: Large-scale MA-AT Scenarios Computation Time

MA-AT	Model	Computation Time (s)
$k_m = 500$, $k_n = 50$	RL(0P)	66.293 (± 19.757)
	RL(0.1P)	75.483 (± 26.232)
	RL(0.5P)	72.756 (± 25.502)
$k_m = 500$, $k_n = 150$	RL(0P)	121.170 (± 37.465)
	RL(0.1P)	121.587 (± 32.632)
	RL(0.5P)	123.796 (± 35.617)

With $k_m = 20$, $k_n = 15$, the 0.5 priority weight model gives the highest average priority over the 50 episodes, and both priority models return over 300% more than the no priority model. Although the no priority model achieves the lowest makespan within its standard deviation, the priority models are on average within the same standard deviation as the no priority. The higher standard deviation for no priority indicates that it has potential to perform significantly better or worse with respect to lowering makespan than the priority models, suggesting that adding priority to the reward reduces some variability. The average wait time has similar behavior where the priority models are within standard deviation of the no priority model due to the no priority model having a high standard deviation.

For the larger scale scenario with $k_n = 50$ and $k_m = 500$, these trends are amplified. The priority models again achieve over 300% improvement in priority rewards, with RL(0.5P) reaching 415.147 compared to RL(0P)’s -172.018. The makespan results show RL(0.1P) actually achieves the lowest mean (104.180), while the substantial variability in RL(0P) (± 30.395 , approximately 35% higher than RL(0.5P)’s ± 22.533) indicates less predictable performance.

Furthermore, we also compare added computation time for the large-scale scenarios. In [7], the original model (no priority) has complexity per decision of $\mathcal{O}(k_n^2 + k_m^2 + k_m k_n)$, while the complexity of the framework for MIP is exponential. For even the smaller scale problem ($k_m = 20$, $k_n = 15$), the state of the art MIP approach CTAS-D [15] took 600 seconds, while their RL approach took 0.62 seconds. Furthermore, the RL approach is compared to a greedy heuristic, which for smaller scenarios has higher computation time, but in the larger scale scenarios ($k_m = 500$, $k_n = 150$), the greedy approach is almost two times slower in computation time [7]. In our work, we compare the computation time of the no priority model to the priority models to determine how adding priority affects the scalability found in [7].

In Table 2, we show that for $k_m = 500, k_n = 50$, the computation time increases modestly when incorporating priority optimization, with approximately 14% and 10% increases from the RL(OP) computation time. For the larger scenario with $k_m = 500, k_n = 150$, the computational overhead of priority models becomes negligible, with all three models performing within 2% of each other (121.170 seconds for RL(OP) compared to 123.796 for RL(0.5P)). The high standard deviations across all models (ranging from ± 19.757 to ± 37.465) indicate substantial overlap in their computation time distributions, suggesting that the priority models maintain the scalability advantages of the original RL approach while adding minimal computational cost.

Finally, the code also generates videos of the workers’ trajectories while completing all the tasks for the small scale $k_m = 20, k_n = 15$ scenarios. Fig 1 illustrates a visual comparison between agents with and without the additional reward term. Our agent tends to complete high-priority tasks (darker color) earlier. We summarize the order of finished tasks in Fig 2.

5 Conclusion

In this work, we implemented the prioritization mechanism to a reinforcement learning-based approach to the Single-Task Robot, Multi-Robot Task Allocation (ST-MR-TA) problem, aiming to bridge the gap between scalable learning methods and the practical need for task prioritization. Building on the species-traits model for heterogeneous teams, we modified the standard objective function, which traditionally minimizes makespan and wasted availability, by introducing a priority reward term, P . Learned via REINFORCE with the baseline and new reward term, our model learns to complete tasks with higher assigned importance values earlier in the schedule, effectively modeling urgency within the reward structure.

Experimental results show formulation successfully altered agent behavior without compromising overall system efficiency. Notably, in large-scale scenarios with 50 agents and 500 tasks, the priority-aware model demonstrated superior stability, achieving the lowest mean makespan and significantly reducing the performance variance observed in the non-prioritized baseline. These findings suggest that incorporating priority constraints can act as a heuristic that stabilizes the learning process in complex, high-dimensional search spaces. In terms of scalability, the difference in computation time between the non-prioritized and prioritized models within our tests is at most an increase of approximately 10 to 14%, indicating that the computational cost of adding priority awareness is not prohibitively expensive in terms of computational resources for this scale.

Looking forward, several directions arise from this work, particularly in extending priority modeling and improving environmental realism.

- **Strict Constraints and Precedence Relations** While our priority-based reward biases execution order, many field applications require hard ordering constraints (e.g., inspection-before-repair, mapping-before-navigation). Extending the framework to incorporate prerequisite completion or multi-stage pipeline dependencies would move the system beyond soft prioritization toward tightly structured mission execution.
- **Dynamic Priorities** Our method assumes fixed task priorities, but real missions require urgency that changes over time. Future work may incorporate dynamic or time-dependent priority functions—potentially informed by external data sources (e.g., weather, wildfire alerts), allowing the system to adapt task importance and respond to evolving conditions.
- **Generalized Priority-Aware MRTA** A natural continuation is scaling toward dynamic and non-stationary environments, where tasks may arrive, expire, or change criticality in real time. Coupling reinforcement learning with predictive models, hierarchical control, or online replanning could enable responsive scheduling for long-horizon deployments.

References

- [1] E. Bischoff, F. Meyer, J. Inga, and S. Hohmann. Multi-robot task allocation and scheduling considering cooperative tasks and precedence constraints. In *2020 IEEE international conference on systems, man, and cybernetics (SMC)*, pages 3949–3956. IEEE, 2020.
- [2] Q. Li, M. Li, B. Q. Vo, and R. Kowalczyk. Distributed near-optimal multi-robots coordination in heterogeneous task allocation. In *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 4309–4314. IEEE, 2020.
- [3] Q. Li, M. Li, B. Q. Vo, and R. Kowalczyk. An anytime algorithm for large-scale heterogeneous task allocation. In *2020 25th international conference on engineering of complex computer systems (ICECCS)*, pages 206–215. IEEE, 2020.
- [4] A. Pal, A. Chauhan, and M. Baranwal. Together we rise: Optimizing real-time multi-robot task allocation using coordinated heterogeneous plays. *arXiv preprint arXiv:2502.16079*, 2025.
- [5] Y. Shida, T. Jimbo, T. Odashima, and T. Matsubara. Reinforcement learning of multi-robot task allocation for multi-object transportation with infeasible tasks. In *2025 IEEE/SICE International Symposium on System Integration (SII)*, pages 1548–1555. IEEE, 2025.
- [6] B. Park, C. Kang, and J. Choi. Cooperative multi-robot task allocation with reinforcement learning. *Applied Sciences*, 12(1):272, 2021.
- [7] W. Dai, U. Rai, J. Chiun, C. Yuhong, and G. Sartoretti. Heterogeneous multi-robot task allocation and scheduling via reinforcement learning. *IEEE Robotics and Automation Letters*, 2025.
- [8] K. Leahy, Z. Serlin, C.-I. Vasile, A. Schoer, A. M. Jones, R. Tron, and C. Belta. Scalable and robust algorithms for task-based coordination from high-level specifications (scratches). *IEEE Transactions on Robotics*, 38(4):2516–2535, 2021.
- [9] L. Zhang, J. Zhao, E. Lamon, Y. Wang, and X. Hong. Energy efficient multi-robot task allocation constrained by time window and precedence. *IEEE Transactions on Automation Science and Engineering*, 2023.
- [10] B. P. Gerkey and M. J. Matarić. A formal analysis and taxonomy of task allocation in multi-robot systems. *The International journal of robotics research*, 23(9):939–954, 2004.
- [11] T. Bektas. The multiple traveling salesman problem: an overview of formulations and solution procedures. *omega*, 34(3):209–219, 2006.
- [12] K. Braekers, K. Ramaekers, and I. Van Nieuwenhuyse. The vehicle routing problem: State of the art classification and review. *Computers & industrial engineering*, 99:300–313, 2016.
- [13] A. Khamis, A. Hussein, and A. Elmogy. Multi-robot task allocation: A review of the state-of-the-art. *Cooperative robots and sensor networks 2015*, pages 31–51, 2015.
- [14] Z. Pan and Y. Yu. Optimizing heterogeneous multi-robot team composition for long-horizon construction tasks: Time-and utilization-guided simulation. *Automation in Construction*, 165: 105520, 2024.
- [15] B. Fu, W. Smith, D. M. Rizzo, M. Castanier, M. Ghaffari, and K. Barton. Robust task scheduling for heterogeneous robot teams under capability uncertainty. *IEEE Transactions on Robotics*, 39(2):1087–1105, 2022.
- [16] G. A. Korsah, B. Kannan, B. Browning, A. Stentz, and M. B. Dias. xbots: An approach to generating and executing optimal multi-robot plans with cross-schedule dependencies. In *2012 IEEE International Conference on Robotics and Automation*, pages 115–122. IEEE, 2012.

- [17] W. Babincsak, A. Aswale, and C. Pincioli. Ant colony optimization for heterogeneous coalition formation and scheduling with multi-skilled robots. In *2023 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)*, pages 121–127. IEEE, 2023.
- [18] J. Goossens, S. Funk, and S. Baruah. Priority-driven scheduling of periodic task systems on multiprocessors. *Real-time systems*, 25(2):187–205, 2003.
- [19] V. C. Kalempa, L. Piardi, M. Limeira, and A. S. de Oliveira. Multi-robot preemptive task scheduling with fault recovery: A novel approach to automatic logistics of smart factories. *Sensors*, 21(19):6536, 2021.
- [20] M. Gombolay, R. Wilcox, and J. Shah. Fast scheduling of multi-robot teams with temporospatial constraints. 2013.
- [21] J. Zhang, Q. Jia, S. Zhang, and G. Chen. Dynamic and prioritized task scheduling of heterogeneous multi-robot systems using deep reinforcement learning. *Neurocomputing*, 638:130184, 2025.
- [22] W. Gao, Z. Yu, H. Xiong, B. Guo, L. Wang, and Y. Yao. Parallel task scheduling in autonomous robotic systems: An event-driven multimodal prediction approach. In *Proceedings of the 53rd International Conference on Parallel Processing*, pages 742–751, 2024.
- [23] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.
- [24] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, Y. Gwon, and S. Min. Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems*, 33:21188–21198, 2020.